Universität
Stuttgart

Coupling fortran routines with OF

- a new ODE solver using Newton Linearization -

Andreas Kronenburg

Acknowledgements: Dr. F. Salehi

Institut für
Technische
Verbrennung

Calling Fortran routines from a C++ program

- The extern “C” directive
  - The extern “C” linkage specification must be used to declare any modules that are not written in C++.
- Case sensitivity
  - C++ is a case sensitive language, Fortran is not.
  - All reference to Fortran symbols must be specified in lowercase in C++ calls.
- The underscore(_)
  - In many Fortran compilers, an underscore (_) is appended by default at the end of definitions and references. The appended underscore is a standard for HEP Fortran libraries. In a C++ call the Fortran subroutines or functions must be specified with an appended underscore.

Calling Fortran routines from a C++ program

- Passing parameters
  - C++ passes all parameters by value (except arrays and structures). Fortran passes them by reference.
  - It is necessary to specify in the C++ function prototypes that the Fortran subroutines and functions arguments using the address-of (&) operator.
- Arrays
  - C++ stores arrays in row-major order, whereas Fortran stores arrays in column-major order.
  - The lower bound for C++ is 0, for Fortran is 1.
- Fortran Subroutines
  - C++ does not distinguish between subroutines and functions. All subprograms are treated as functions. Fortran subroutines can be seen as a void function (the functions do not return a value).

Example: ODE solver using Newton Linearization

To solve: $\frac{dY_i}{dt} = \dot{\omega}_i$ or for concentrations $\frac{dc_i}{dt} = r_i$

For system: $$\left(\frac{\partial c_i}{\partial t}\right) = \sum_{r=1}^{R}\left(\nu_{ri}^{prod} - \nu_{ri}^{reac}\right)\left(k_r^f \prod_{s=1}^{S} c_s^{\nu_{rs}^{prod}} - k_r^r \prod_{s=1}^{S} c_s^{\nu_{rs}^{reac}}\right) = r_i; \quad i = 1, S$$

Discretize $$\frac{c_i^{n+1} - c_i^n}{\partial t} = r_i^{n+1} = r_i^n + \frac{\partial r_i}{\partial c_j}\left(c_j^{n+1} - c_j^n\right)$$

Now solve $$\left(\frac{1}{\Delta t} I - A_{ij}\right) c_j^{n+1} = \frac{c_i^n}{\Delta t} + r_i^n - A_{ij} c_j$$

with $$A_{ij} = \sum_{k=1}^{nreac} \nu_{ik} \frac{\partial R_k}{\partial c_j}$$

R is reaction rate (forward and backward combined for simplicity), I is unity matrix and ν is stoichiometric coefficient

New chemistry solver for stiff odes

Two options:

1. as option that can be selected as solver in subdictionary odeCoeffs in dictionary constant/chemistryProperties

   ⟶ would require user library linked to OF in system/controlDict

2. as submodel for reactivePopeParticle that can be selected as „reactionModel“ in dictionary constant/cloudProperties

Choose option 2 since option 1 does not offer simple interface for fortran routines

OF Modifications

Add option of „finiteRateNewtonLinODE“ submodel in

src/lagrangian/mmc/popeParticles/include/makeReactingPopeParticleReactionModels.H

```
\*---------------------------------------------------------------------------*/

#ifndef makeReactingPopeParticleReactionModels_H
#define makeReactingPopeParticleReactionModels_H

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "NoReaction.H"
#include "finiteRateParticleReaction.H"
#include "flameSheetParticleReaction.H"
#include "finiteRateNewtonLinODE.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#define makeReactingPopeParticleReactionModels(CloudType)                    \
                                                                              \
    makeReactionModel(CloudType);                                             \
    makeReactionModelType(NoReaction, CloudType);                             \
    makeReactionModelType(finiteRateParticleReaction, CloudType);             \
    makeReactionModelType(flameSheetParticleReaction, CloudType);  \
    makeReactionModelType(finiteRateNewtonLinODE, CloudType);



// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif
```

OF Modifications

- **Create new directory**

src/lagrangian/mmc/submodels/Reacting/ReactionModel/finiteRateNewtonLinODE/

- **Create new file finiteRateNewtonLinODE.H**

1. copy finiteRateParticleReaction.H
2. replace within this file all instances of *finiteRateParticleReaction* by *finiteRateNewtonLinODE*

OF Modifications

- **Create new file finiteRateNewtonLinODE.C**

1. copy finiteRateParticleReaction.C
2. replace within this file all instances of *finiteRateParticleReaction* by *finiteRateNewtonLinODE*
3. Modify constructor – need to initialize the F77 ODE solver calling fortran routine *newtonlinodeinit_*

```
// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * * //

template<class CloudType>
Foam::finiteRateNewtonLinODE<CloudType>::finiteRateNewtonLinODE
(
    const dictionary& dict,
    CloudType& owner
)
:
    ode
    <
        popeParticleChemistryModel
        <
            basicParticleChemistryModel,
            gasHThermoPhysics
        >
    >(owner.mesh()),

    ReactionModel<CloudType>(dict,owner,typeName),

        zLower_(this->coeffDict().lookupOrDefault<scalar>("zLower", 0)),

        zUpper_(this->coeffDict().lookupOrDefault<scalar>("zUpper", 1))
{
    newtonlinodeinit_
    (
    );

}
```

OF Modifications

4. Modify function *calculate* – need to call F77 ODE solver calling fortran routine *newtonlinodesolve_* (routine called for every PopeParticle)

```
template<class CloudType>
void Foam::finiteRateNewtonLinODE<CloudType>::calculate
(
    scalar t0,
    scalar dt,
    scalar& ha,
    scalar pc,
    scalar& pT,
    scalar pz,
    scalarField& Y
)
{
    if (pz < zLower_ || pz > zUpper_)
    {
        return;
    }

    label nspec = Y.size();
    scalar  *pSpec=&Y[0];
    scalar Temp = pT;
    scalar pascalToCGS = 10.;
    scalar P_dynes = pc * pascalToCGS;
    scalar deltat = dt;

    newtonlinodesolve_
    (
        pSpec,
        &nspec,
        &Temp,
        &P_dynes,
        &deltat
    );
}
```

OF Modifications

- **Create new file newtonLinODEExtern.H**

need to define fortran routines as external function with *extern "C"*

```
  Description
  External fortran functions

  SourceFiles
  finiteRateNewtonLinODE.C

*---------------------------------------------------------------------------*/
namespace Foam
{

  // External functions (fortran)

    extern "C"
    {
        void newtonlinodeinit_
        (
        );

        void newtonlinodesolve_
        (
            scalar* Ypart,
            label* nSpec,
            scalar* Tpart,
            scalar* Ppart,
            scalar* dt
        );

    }
}

// ************************************************************************* //
```

OF Modifications

- **Add in** src/lagrangian/mmc/Make/options **the fortran library**

```
LIB_LIBS = \
        -lNewtonLinF90
```

- **Create the fortran library in mmcThirdParties-2.4.x**
  - Execute the makefile in directory NEWTONLIN

```
LIBS=-lm

#CF=gfortran
CF=f95

DESTLIBDIR=${FOAM_USER_LIBBIN}
DESTBINDIR=${FOAM_USER_APPBIN}

OBJ = ChemistryModule_CK.o initNewtonLinODE_CKPLG.o NewtonLinODE_CKPLG.o cklib_plg.o common_CKPLG.o numrecipes.o tranlib.o auxil.o math.o amach.o

#all: shared program install clean
all: shared install clean

%.o: %.F90 ; $(CF) -c -fPIC -fdefault-real-8 -fdefault-double-8 -ffree-line-length-none ChemistryModule_CK.F90 initNewtonLinODE_CKPLG.F90
NewtonLinODE_CKPLG.F90 cklib_plg.f common_CKPLG.f numrecipes.f tranlib.f auxil.f math.f amach.f

shared: $(OBJ) ; $(CF) -shared -o libNewtonLinF90.so *.o

#program: $(OBJ) ; $(CF) -o solveChem *.o

install: ; cp libNewtonLinF90.so $(DESTLIBDIR) #; cp solveChem $(DESTBINDIR)

clean: ; rm -f *.o *.mod *~
```

Fortran Routines

- **key fortran routines**

1) *initNewtonLinODE_CKPLG.F90* contains

subroutine NEWTONLINODEINIT

initializes the ode solver, reads in the chemkin files, i.e. does require existence of linkfile.d and tranlink.d. Ideally, this could be avoided by passing the relevant data from OF to the corresponding fortran array since chemkin files are also read in by the OF chemkinReader. Future development may avoid this redundancy.

2) *NewtonLinODE_CKPLG.F90* contains

subroutine NEWTONLINODESOLVE(spec0,nspec,Temp,press,dtime)

ode integration for every point particle (requires arguments passed from OF)

Fortran Routines

Careful: Some of the third party libraries use confidential snippets of source code

→ Need to create dummy libraries to allow for a consistent Make/options file even if fortran source code is not provided

- Create folder in *mmcThirdParty-2.4.x/DUMMY_Libs*
- Create *makefile*

```
LIBS=-lm

#CF=gfortran
CF=f95

DESTLIBDIR=${FOAM_USER_LIBBIN}
DESTBINDIR=${FOAM_USER_APPBIN}

OBJ = dummyNewtonLinODE.o

#all: shared program install clean
all: shared install clean

#%.o: %.F90 ; $(CF) -c -fPIC -fdefault-real-8 -fdefault-double-8 -ffree-line-length-none dummyNewtonLinODE.F90
%.o: %.F90 ; $(CF) -c -fPIC dummyNewtonLinODE.F90

shared: $(OBJ) ; $(CF) -shared -o libNewtonLinF90.so *.o

install: ; cp libNewtonLinF90.so $(DESTLIBDIR)

clean: ; rm -f *.o *.mod *~
```

- Create routine *dummyNewtonLinODE.F90* with dummy subroutines *NEWTONLINODEINIT* and *NEWTONLINODESOLVE* that print clear error statements

- **Tutorial case in**

  flameD/NewtonLinODE

- **Select ode solver in control/cloudProperties**

```
reactionModel finiteRateNewtonLinODEChemistry;

finiteRateNewtonLinODEChemistryCoeffs
{
   zLower          0.1;
   zUpper          0.9;
}
```

Complete list of options:

- no other coefficients can be specified

Further input needed:

- linkfile.d (binary chemkin file must be in folder *./chemkin/*)
- tranlink.d (binary chemkin file must be in folder *./chemkin/*)

Results

- **Performance**
  - at least 2x slower than rodas23
  - only use if uncertain of ODE stiff equation solver performance
  - may not work for non-elementary reaction equations (not tested)

IDEALLY: Here should come some test results

Guidelines

1. Think about structure – discuss in OF seminars
2. No separate solver for each problem
3. Do not modify existing code or KEEP TO MINIMUM
4. Stick to modular programming – use existing libraries where possible
5. Stick to OF syntax
6. Existing code must still run with old/test problems (to be defined)
7. Documentation
   a) e.g. powerpoint
   b) should contain
      - Intro
      - Document structure
      - OF snippets – step-by-step tutorial on how changes where effectuated
      - Tutorial and its location
      - Results (thorough testing)
      - Performance